

ADVERTISEMENT

Python Iterators

[< Previous](#)[Next >](#)

Python Iterators

An iterator is an object that contains a countable number of values.

An iterator is an object that can be iterated upon, meaning that you can traverse through all the values.

Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods `__iter__()` and `__next__()`.

Iterator vs Iterable

Lists, tuples, dictionaries, and sets are all iterable objects. They are iterable *containers* which you can get an iterator from.

All these objects have a `iter()` method which is used to get an iterator:

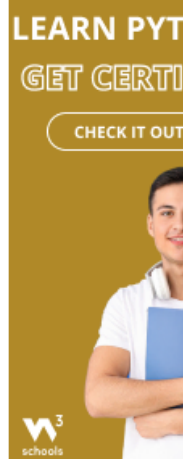
Example

[Get your own Python Server](#)

Return an iterator from a tuple, and print each value:

```
mytuple = ("apple", "banana", "cherry")
myit = iter(mytuple)

print(next(myit))
print(next(myit))
print(next(myit))
```

[Try it Yourself »](#)

COLOR
PICKER





Even strings are iterable objects, and can return an iterator:

Example

Strings are also iterable objects, containing a sequence of characters:

```
mystr = "banana"
myit = iter(mystr)
```

[Tutorials ▾](#)[References ▾](#)[Exercises ▾](#)[Certificates ▾](#)[Sign In](#)[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C++](#) [C#](#) [BOOT!](#) [➤](#)

- Python Lists ▾
- Python Tuples ▾
- Python Sets ▾
- Python Dictionaries ▾
- Python If...Else
- Python Match
- Python While Loops
- Python For Loops
- Python Functions
- Python Decorators
- Python Range
- Python Lambda
- Python Arrays
- Python OOP
- Python Classes/Objects
- Python Inheritance
- Python Iterators
- Python Polymorphism
- Python Scope
- Python Modules
- Python Dates
- Python Math
- Python JSON
- Python RegEx
- Python PIP
- Python Try...Except
- Python String Formatting
- Python None
- Python User Input
- Python VirtualEnv

File Handling

- Python File Handling
- Python Read Files

```
print(next(myit))
print(next(myit))
print(next(myit))
```

[Try it Yourself »](#)

Looping Through an Iterator

We can also use a [for](#) loop to iterate through an iterable object:

Example

Iterate the values of a tuple:

```
mytuple = ("apple", "banana", "cherry")

for x in mytuple:
    print(x)
```

[Try it Yourself »](#)

Example

Iterate the characters of a string:

```
mystr = "banana"

for x in mystr:
    print(x)
```

[Try it Yourself »](#)

AC

R

The `for` loop actually creates an iterator object and executes the `next()` method for each loop.

ADVERTISEMENT

[REMOVE ADS](#)

Create an Iterator

To create an object/class as an iterator you have to implement the methods `__iter__()` and `__next__()` to your object.

As you have learned in the [Python Classes/Objects](#) chapter, all classes have a function called `__init__()`, which allows you to do some initializing when the object is being created.

The `__iter__()` method acts similar, you can do operations (initializing etc.), but must always return the iterator object itself.

The `__next__()` method also allows you to do operations, and must return the next item in the sequence.

Example

Create an iterator that returns numbers, starting with 1, and each sequence will increase by one (returning 1,2,3,4,5 etc.):

```
class MyNumbers:
    def __iter__(self):
        self.a = 1
        return self

    def __next__(self):
        x = self.a
        self.a += 1
        return x

myclass = MyNumbers()
myiter = iter(myclass)

print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
```

Try it Yourself »

StopIteration

The example above would continue forever if you had enough next() statements, or if it was used in a `for` loop.

To prevent the iteration from going on forever, we can use the `StopIteration` statement.

In the `__next__()` method, we can add a terminating condition to raise an error if the iteration is done a specified number of times:

Example

Stop after 20 iterations:

```
class MyNumbers:
    def __iter__(self):
        self.a = 1
        return self

    def __next__(self):
        if self.a <= 20:
            x = self.a
            self.a += 1
            return x
        else:
            raise StopIteration

myclass = MyNumbers()
myiter = iter(myclass)

for x in myiter:
    print(x)
```

Try it Yourself »

Exercise ?

There are two methods that you have to implement when you create an iterator, which two?

- ☐ `__iter__()` and `__next__()`
- ☐ `__next__()` and `__prev__()`
- ☐ `__init__()` and `__end__()`

Submit Answer »

◀ Previous

Sign in to track progress

Next ▶

ADVERTISEMENT

ADVERTISEMENT